Profiling and Performance Tuning

Objectives

After completing this lab, you will be able to:

- Setup the board support package (BSP) for profiling an application
- Set the necessary compiler directive on an application to enable profiling
- Setup the profiling parameters
- Profile an application and analyze the output

Steps

Create a Vivado Project

Launch Vivado and create an empty project, called lab6, targeting the PYNQ-Z1 or PYNQ-Z2 boards and using the Verilog language.

1. Open Vivado and create a new project new project call *lab6* in the **{labs}** directory.
2. Select the **RTL Project** option in the *Project Type* form, and click **Next**.
3. Select **Verilog** as the *Target Language* in the *Add Sources* form, and click **Next**.
4. Click **Next** two times.
5. In the *Default Part* form, click on *Boards* filter and Select **www.digilentinc.com** for the *PYNQ-Z1* board, **tul.com.tw** for the *PYNQ-Z2* board in the Vendor field, select *PYNQ-Z1__or pynq-z2*, and click **Next**.
6. Click **Finish** to create an empty Vivado project.

Set the project settings to include provided fir_top IP

1. Click **Settings** in the *Flow Navigator* pane.
2. Expand **IP** in the left pane of the *Project Settings* form.
3. Click Repository and using "minus" button remove entries, if any.
4. Click on the "plus" button, browse to **{sources}\lab6** and click** Select**.
5. Click **OK**.

   The directory will be scanned and it will report one IP was detected.
6. Click **OK** twice.

Creating the Hardware System Using IP Integrator

Create a block design in the Vivado project using IP Integrator to generate the Zynq ARM Cortex-A9 processor based hardware system.

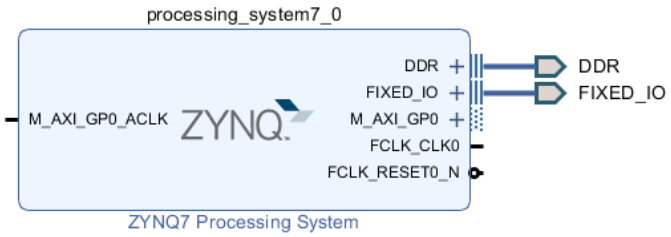
1. In the Flow Navigator, click **Create Block Design** under IP Integrator.

2. Name the block **system** and click **OK**.
3. Click on the + button.
4. Once the IP Catalog is open, type **zy** into the Search bar, and double click on **ZYNQ7 Processing System** entry to add it to the design.
5. Click *Run Block Automation*,and click **OK** to accept the default settings.
6. Double click on the Zynq block to open the *Customization* window for the Zynq processing system.

   A block diagram of the Zynq should now be open, showing various configurable blocks of the Processing System.

Configure the I/O Peripherals block to only have UART 0 support.

1. Click on the *MIO Configuration* panel to open its configuration form.
2. Expand the *I/O Peripherals* on the right.
3. Uncheck *ENET 0*, *USB 0*, and *SD 0*, *GPIO (GPIO MIO)*, leaving *UART 0* selected.
4. Click **OK**.



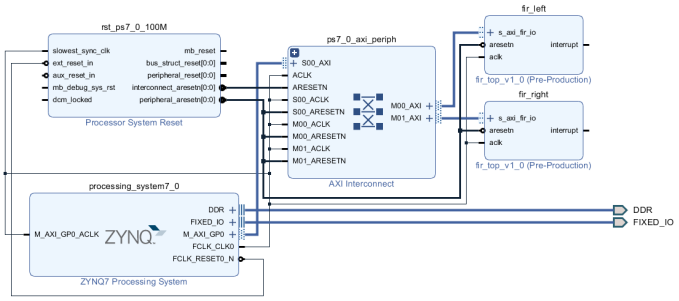
ZYNQ Processing System configured block

Add FIR Core to the System

Instantiate the provided FIR core twice naming the instances as fir_left and fir_right. Validate the design.

1. Click the + button and search for **fir** in the catalog.
2. Double-click on the **fir_top_v1_0** to add the IP instance to the system
3. Select the *fir_top_1* instance and change its name to **fir_left** in its property form.
4. Click the + button and search for **fir** in the catalog.
5. Double-click on the **fir_top_v1_0** to add the IP instance to the system
6. Select the *fir_top_1* instance and change its name to **fir_right** in its property form.
7. Click on **Run Connection Automation** , and select **All Automation** to select *fir_left* and *fir_right* **.**
8. Click on *s_axi_fir_io* for both *fir_left* and *fir_right* and confirm that they will be automatically connected to the Zynq *M_AXI_GP0* port
9. Click **OK** toconnect the two blocks to the *M_AXI_GP0*

The design should look similar to shown below:



The completed design

It is not necessary to connect the *interrupt* signals of the *fir* blocks.

10. Select the *Diagram* tab, and click on the (Validate Design) button to make sure that there are no errors.

    Ignore warnings.

Generate the Bitstream

Create the top-level HDL of the embedded system, and generate the bitstream.

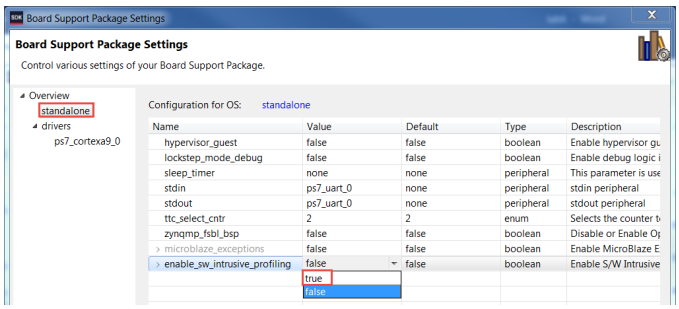
1. In Vivado, select the *Sources* tab, expand the *Design Sources,* right-click the *system.bd* and select **Create HDL Wrapper** and click **OK**.
2. Click on the **Generate Bitstream** in the *Flow Navigator* pane to synthesize and implement the design, and generate the bitstream.
3. Click **Save** to save the design and **Yes** to run the necessary processes. Click **OK** to launch the runs.
4. When the bitstream generation process has completed click **Cancel.**

Export the Design to the SDK

Export the design to the SDK, create the software BSP using the standalone operating system and enable the profiling options.

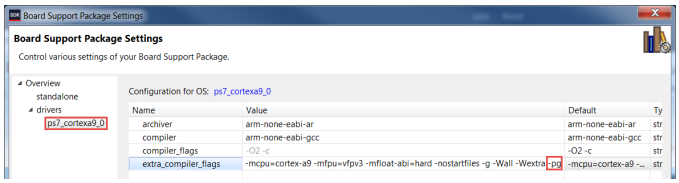
1. Export the hardware configuration by clicking **File > Export > Export Hardware...**
2. Tick the box to *Include Bitstream*, and click **OK**
3. Launch SDK by clicking **File > Launch SDK** and click **OK**
4. In SDK, select **File** > **New** > **Board Support Package.**
5. Notice **Standalone_bsp_0** in the **Project name** field and click **Finish** with default settings.

   A Board Support Package Settings window will appear.
6. Select the **Overview > standalone** entry in the left pane, click on the drop-down arrow of the *enable_sw_intrusive_profiling Value* field and select **true**.



Enable profiling in the board support package

7. Select the **Overview > drivers > cpu_cortexa9** and add **–pg** in the *extra_compiler_flags__Value* field.



Adding profiling switch

8. Click **OK** to accept the settings and create the BSP.

Create the Application

Create the *lab6* application using the provided lab6.c, fir.c, fir.h, fir_coef.dat, and xfir_fir_io.h files.

1. Select **File** > **New** > **Application Project.**
2. Enter **lab6** as the project name, select the **Use existing** *standalone_bsp_0* option, and click **Next**.
3. Select **Empty Application** in the *Available Templates* pane and click **Finish**.
4. In the *lab6* project, right click on the *src* directory and select **Import.**
5. Expand the General folder and double-click on **File system,** and browse to the **{sources}\lab6** directory.
6. Select **fir_coef.dat, fir.c, fir.h, lab6.c,** and **xfir_fir_io.h,** and click **Finish.**

   The program should compile successfully and generate the lab6.elf file.
7. Open the *lab6.c* file and scroll to the main function at the bottom. Notice the following code:

```
#ifdef SW_PROFILE
    fir_software(&output,signal);
#else
    filter_hw_accel_input(&output,signal);
#endif
```

Source code snippet

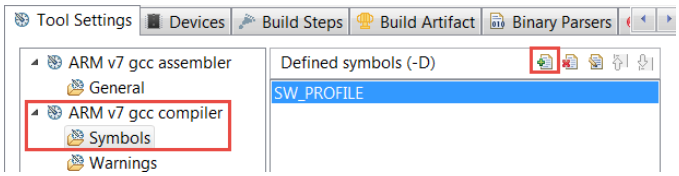
The function *fir_software*( ) function is a software implementation of the FIR function. The *filter_hw_accel_input*( ) function offloads the FIR function to the two FIR blocks that have been implemented in the PL.

Run the Application and Profile

Place the board into the JTAG boot up mode. Program the PL section and run the application using the user defined SW_PROFILE symbol.

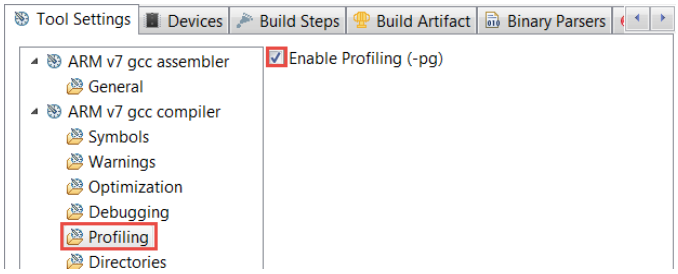
1. Place the board in the JTAG boot up mode.
2. Power ON the board.
3. Select **Xilinx > Program FPGA** and click on **Program**.
4. Right click on the *lab6 directory*, and select **C/C++ Build Settings**.
   5.
5. Under the **ARM v7 gcc compiler** group, select the **Symbols** sub-group **,** click on the alt tag button to open the value entry form, enter **SW_PROFILE** , and click **OK**.

   This will allow us to profile the software loop of the FIR application.



Add user-defined symbol

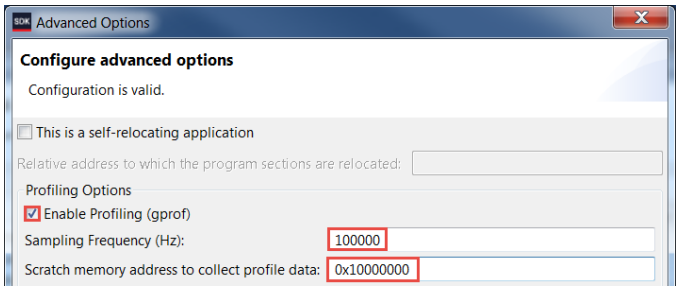
6. Under the **ARM v7 gcc compiler** group, select the **Profiling** sub-group, then check the **Enable Profiling** box, and click **OK**.



Compiler setting for enabling profiling

7. From the menu bar, Select **Run > Run Configurations...** and double click on *Xilinx C/C++ application (System Debugger)* to create a new configuration.
8. Click on the newly created **lab6 Debug** configuration, and select the **Application** tab.

9. Click on the *Advance Options* **Edit...** button.
10. Click on the *Enable Profiling (gprof)* check box, enter **100000** (100 kHz) in the Sampling Frequency field, enter **0x10000000** in the scratch memory address field, and click **OK**.



Profiling options

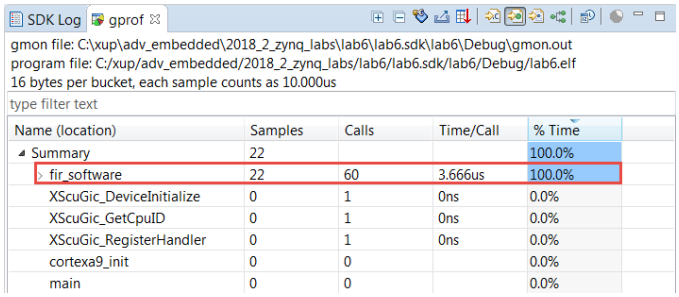
11. Click **Apply** and then click the **Run** button to download the application and execute it.

    The program will run.

Analyze the results.

1. When execution is completed, the Gmon File Viewer dialog box will appear showing *lab6.elf* as the corresponding binary file. Click **OK**.
2. Click on the **Sort samples per function** button ().
3. Click in the **%Time** column to sort in the descending order.

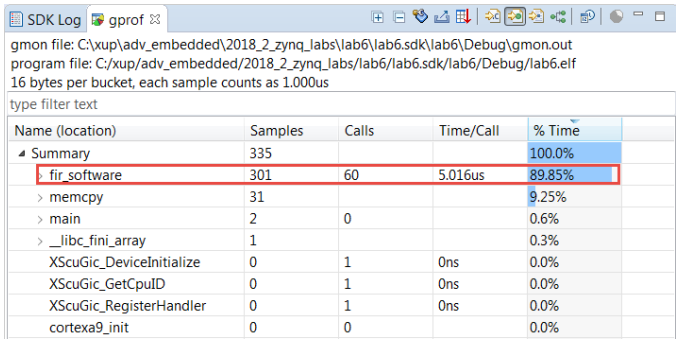
   Note that the fir_software routine is called 60 times, 22 samples were taken during the profiling, and on an average of 3.333 (PYNQ-Z1) or 3.666 (PYNQ-Z2/PYNQ-Z2)microseconds were spent per call.



Sorting results

4. Go back to the *Run Configuration*, and change the sampling frequency to **1000000** (1 MHz) and profile the application again.
5. When execution is completed, click **OK** and the gprof viewer will be updated.
6. Invoke **gprof** , select the **Sorts samples per function** output, and sort the **%Time** column.

Notice that the output has better resolution and reports more functions and more samples per function calls. Note that the number of calls to the fir_software function has not changed but the number of samples taken increased, and the average time spent per call is 5.250 (PYNQ-Z1) or 5.016 (PYNQ-Z2) microseconds in the figure below.



Profiled results with 1 MHz sampling frequency

At this stage, the designer of the system would decide if the FIR function should be ported to hardware.

Profile the application using the hardware FIR filter IP by removing the user defined SW_PROFILE symbol.

1. Select the *lab6* application, right-click, and select **C/C++ Build Settings**.
2. Under the **ARM v7 gcc compiler** group, select the **Symbols** sub-group **,** select **SW_PROFILE** , and delete it by clicking on the delete button.

   This will allow us to profile the hardware IP of the FIR application.
3. Click **Apply** , and then click **OK**
4. Select **Run > Run Configurations** and click the **Run** button to profile the application again and click **OK** when profiling completes.

   Notice that the output now shows filter_hw_accel_input function call instead of the fir_software function call. Note that the average time spent per call is much less as the filtering is done in the hardware instead of the software.
5. Close the SDK and Vivado programs by selecting **File > Exit** in each program.
6. Turn OFF the power on the board.

Conclusion

This lab led you through enabling the software BSP and the application settings for the profiling. You went through creating the hardware which included the hardware IP and was later profiled in the application. You analyzed the profiled application output.